

★ WEEK 12 · PART 3 of 3

Discretization, Calibration, and Pricing

Making the Models Run and Match Reality

Euler-Maruyama · Milstein · calibration · pricing by simulation

To use a short-rate model you must run it on a computer and fit it to data. We discretize the SDEs, calibrate the parameters, and price a bond end-to-end by simulation — checked against the closed form.

3

PART 3 · COMPUTATION

From SDE to a Priced Bond

Discretizing the SDEs with Euler-Maruyama and Milstein, the pitfalls to avoid, calibrating to market data, and pricing a bond by simulation — the full computational workflow.

From Formulas to Working Code

Parts 1-2 gave the theory; Part 3 runs it

1

We have models and formulas

Vasicek and CIR, and closed-form bond prices and yields. But real work needs to simulate, fit, and price numerically.

2

SDEs must be discretized

A computer can't take dt to zero. We approximate the SDE with finite steps — Euler-Maruyama, and the finer Milstein.

3

Then calibrate and price

Fit the parameters to data, simulate paths, and average discounted payoffs into a price — validated against the closed form.

Part 3 in one line: discretize the SDE, calibrate to data, and price a bond by simulation.

Why Discretize?

Computers can't take dt to zero

An SDE is written in infinitesimal time, but a computer works in finite steps. Discretization replaces the continuous dt with a small but finite step, turning the SDE into a recipe a machine can actually run.

The gap

- The SDE lives in continuous time, with dt going to zero.
- A computer must take steps of some finite size.
- We need a rule to advance the state one step.

The pay-off

- Simulate any model, even without a closed form.
- Generate paths for pricing and risk.
- The foundation for Monte Carlo pricing.

Euler-Maruyama

The simplest way to step an SDE

$$\text{Euler-Maruyama: } X_{t+\Delta t} = X_t + a(X_t) \Delta t + b(X_t) \sqrt{\Delta t} Z, \quad Z \sim \mathcal{N}(0, 1)$$

Reading it

- Advance the drift by a term proportional to dt .
- Add a random shock scaled by the root of dt .
- The shock is a standard normal draw.

Why root- dt

- Brownian increments have variance dt .
- So their standard deviation scales as root- dt .
- That is the hallmark of a stochastic step.

Reading the Euler Step

Drift by dt , noise by $\sqrt{dt}$

The drift part

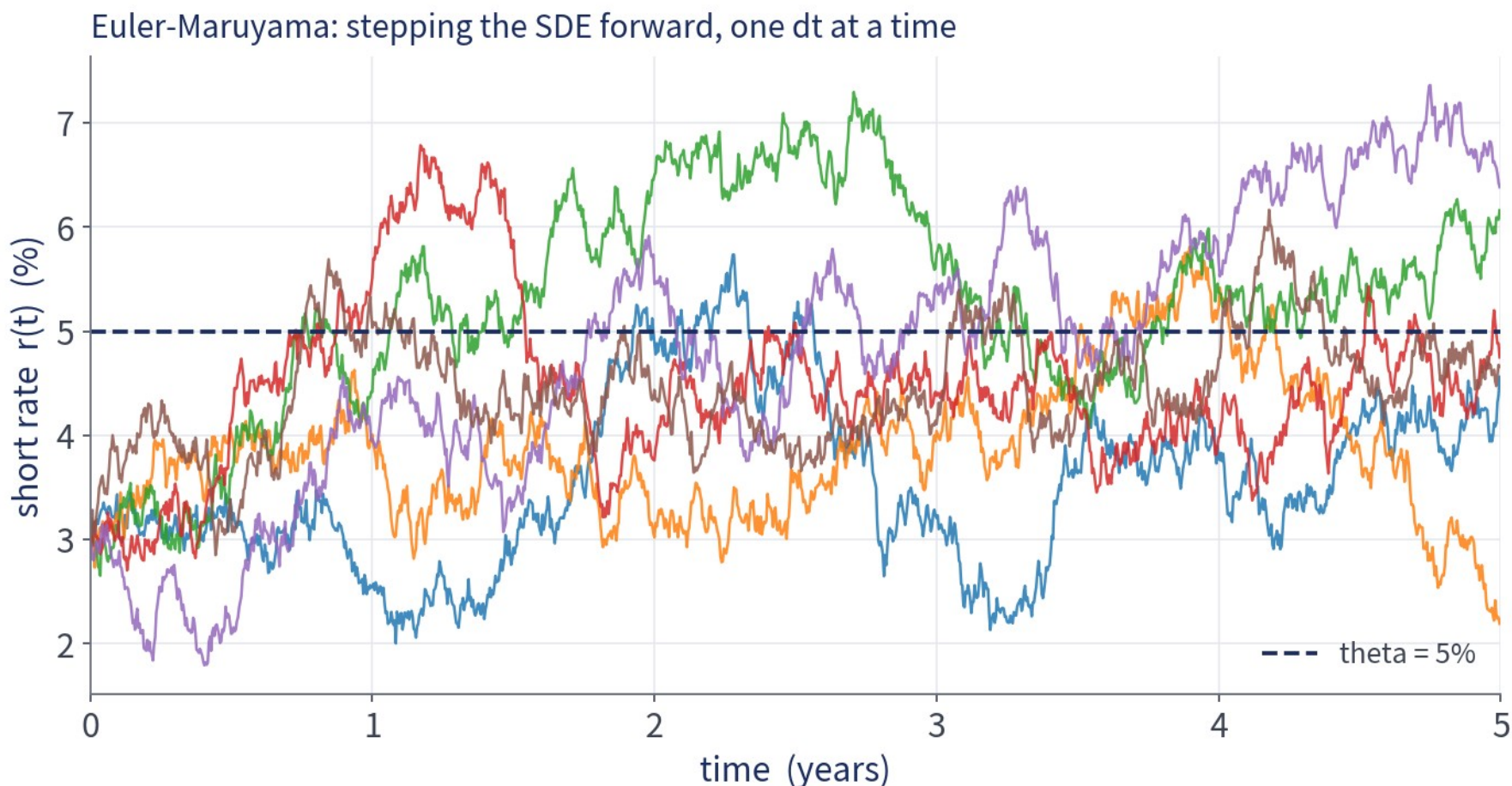
- Deterministic: the trend times the step size.
- For Vasicek, it pulls the rate toward θ .
- Scales linearly with dt .

The diffusion part

- Random: the diffusion times a normal draw.
- Scales with the square root of dt , not dt .
- This is what makes the path jagged.

Euler-Maruyama in Action

Stepping the Vasicek SDE forward one dt at a time produces sample rate paths that revert toward θ . This is the raw material for simulation-based pricing and risk.



Euler's Pitfalls

Discretization error and negative rates

Euler is simple but imperfect. Too large a step introduces discretization error, and for CIR the scheme can push the rate below zero — where the square root is undefined. Both need care.

Discretization error

- A finite step only approximates the true SDE.
- Larger steps mean larger error.
- Shrinking dt improves accuracy — at a cost.

The CIR zero problem

- A big negative shock can drive CIR's rate below zero.
- Then the square root of the rate is undefined.
- Naive Euler breaks exactly where CIR should be safe.

The Negative-Value Fix for CIR

Keeping the square root well-defined

Simple fixes

- Full truncation: use the positive part of the rate inside the root.
- Reflection: bounce a negative value back above zero.
- Floor the rate at zero after each step.

The exact scheme

- CIR has a known transition (non-central chi-squared).
- Sampling from it avoids the problem entirely.
- More work, but exact and always non-negative.

Milstein: One Correction Term

More accuracy for level-dependent diffusion

$$\text{Milstein: } + \frac{1}{2} b(X_t) b'(X_t) [(\sqrt{\Delta t} Z)^2 - \Delta t] \quad \leftarrow \text{extra correction}$$

What it adds

- The Euler step, plus a second-order correction.
- The correction uses the diffusion's derivative.
- It matters most when diffusion depends on the state.

The benefit

- It captures curvature Euler misses.
- It converges faster as the step shrinks.
- For constant-diffusion Vasicek, it reduces to Euler.

Euler vs Milstein

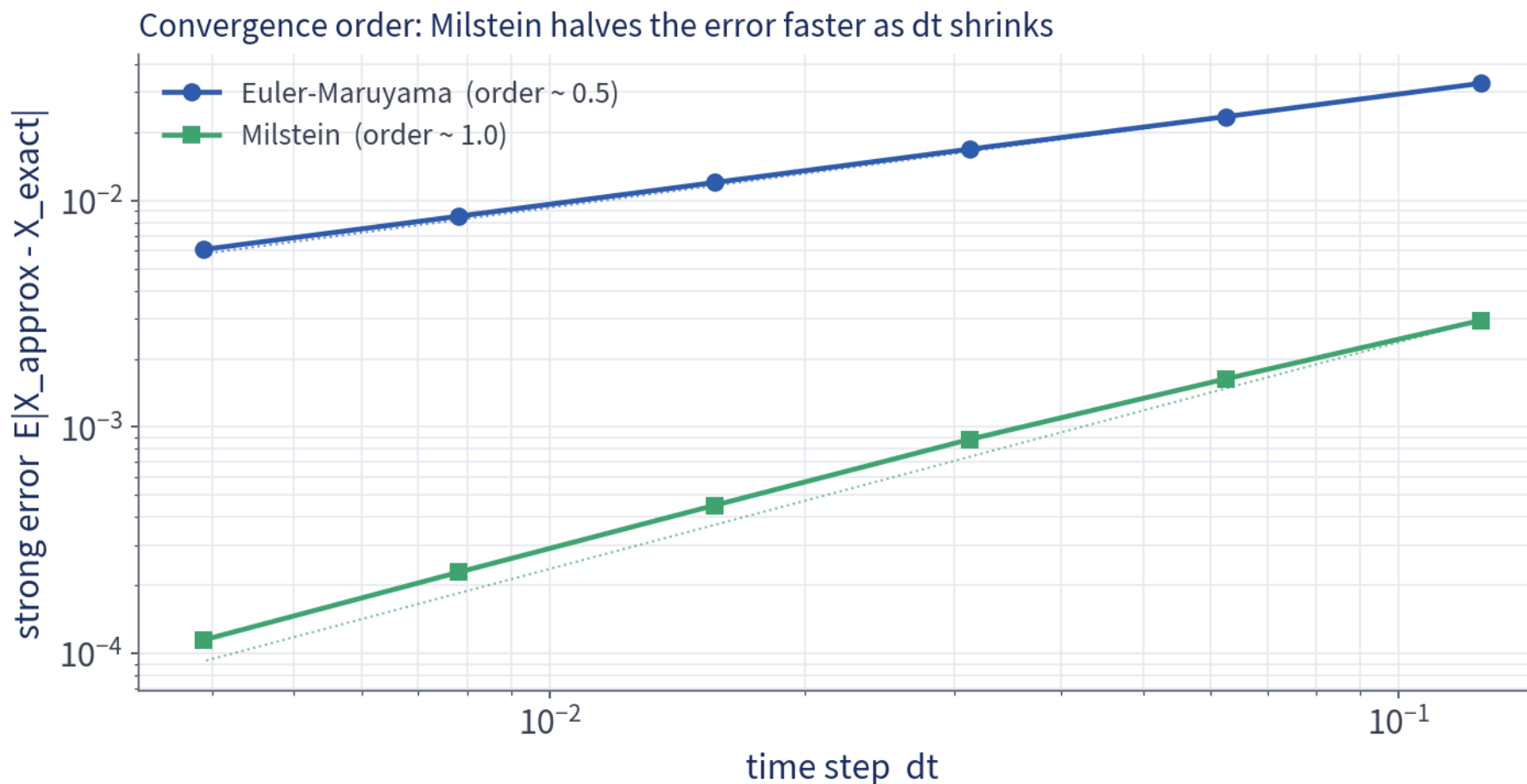
Simplicity against accuracy

Aspect	Euler-Maruyama	Milstein
Update	drift dt + diffusion $\sqrt{dt}$ Z	plus a correction term
Strong order	about 0.5	about 1.0
Needs	only drift and diffusion	also the diffusion's derivative
Cost per step	cheapest	slightly more
When to use	quick default	when accuracy matters

Milstein halves the error faster as dt shrinks — at the price of one extra term.

Convergence Order

Strong error versus step size, on a log-log scale. Euler's error falls with slope about one-half; Milstein's falls with slope about one — it reaches a given accuracy with far coarser steps.



Calibrating to Market Data

Making the model match reality

A model is only useful once its parameters reflect reality. Calibration fits κ , θ , and σ so the model agrees with data — either the historical behavior of rates, or today's observed prices.

The goal

- Choose parameters so the model matches the world.
- Either match how rates have MOVED, or how bonds are PRICED.
- Different data answer different questions.

Two regimes

- Time-series fit: estimate from historical rates.
- Cross-sectional fit: match today's yield curve.
- Sometimes both, for a full picture.

Two Sources of Data

History versus prices

Historical rates

- A time series of the short rate over time.
- Estimate the real-world dynamics: reversion and noise.
- Uses the AR(1) bridge from Week 11.

Market prices

- Today's observed bond prices or yields.
- Fit parameters so the model curve matches them.
- Gives the risk-neutral parameters for pricing.

Estimating From History

The AR(1) bridge, one more time

from history: regress r_{t+1} on $r_t \Rightarrow \phi \Rightarrow \kappa = -\frac{\ln \phi}{\Delta t}, \theta, \sigma$

The recipe

- Regress the rate on its own one-step lag.
- The slope gives the persistence ϕ .
- Convert ϕ into κ , θ , and σ .

Why it works

- Discretized Vasicek is exactly an AR(1).
- So the same estimator from Week 11 applies.
- A one-line regression yields all three parameters.

Calibrating to the Curve

Matching today's observed yields

The procedure

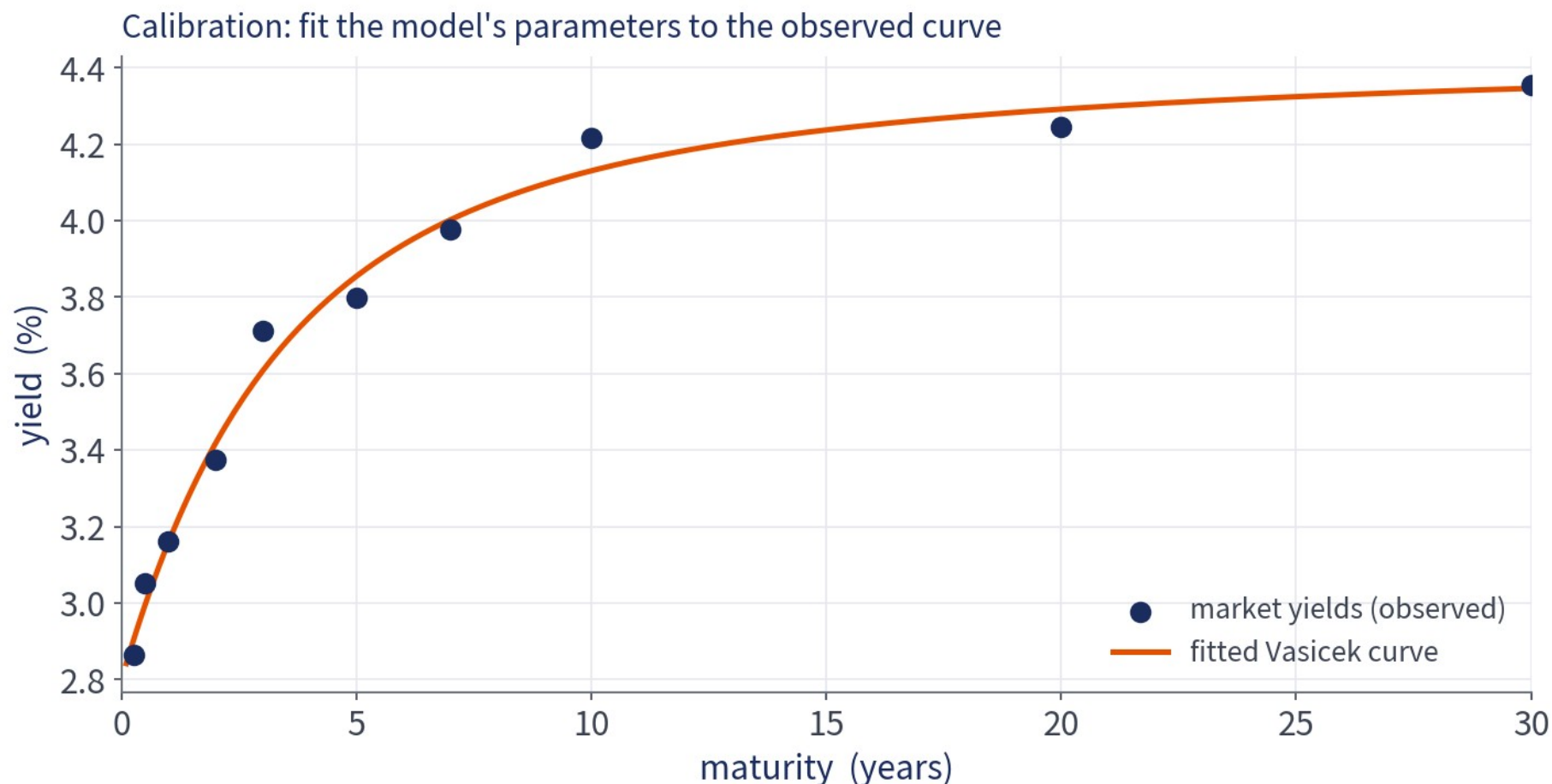
- Take the observed yields across maturities.
- Search for parameters whose model curve fits them.
- Minimize the gap between model and market.

Why it differs from history

- It targets prices, not past dynamics.
- It gives the parameters used for pricing.
- The two fits can, and often do, differ.

Calibration to the Yield Curve

Choosing the parameters so the model's yield curve passes through the observed market yields. A good fit means the model reproduces today's prices before you trust it for anything else.



Pricing a Bond by Simulation

The end-to-end approach

With a calibrated model you can price by simulation: generate many short-rate paths, discount along each, and average. For Vasicek and CIR you can check the answer against the closed form — then trust the engine on harder products.

Why simulate at all

- Many real products have no closed-form price.
- Simulation handles them uniformly.
- The bond is the simplest case to validate on.

The validation

- Price the bond both ways: simulation and formula.
- They should agree as paths increase.
- Agreement builds trust in the simulator.

The Simulation Workflow

Simulate, discount, average

1

Simulate paths

Step the calibrated SDE forward with Euler or Milstein to generate many short-rate paths to the bond's maturity.

2

Discount each path

Along each path, accumulate the rate over time and form its discount factor, the exponential of minus that accumulated rate.

3

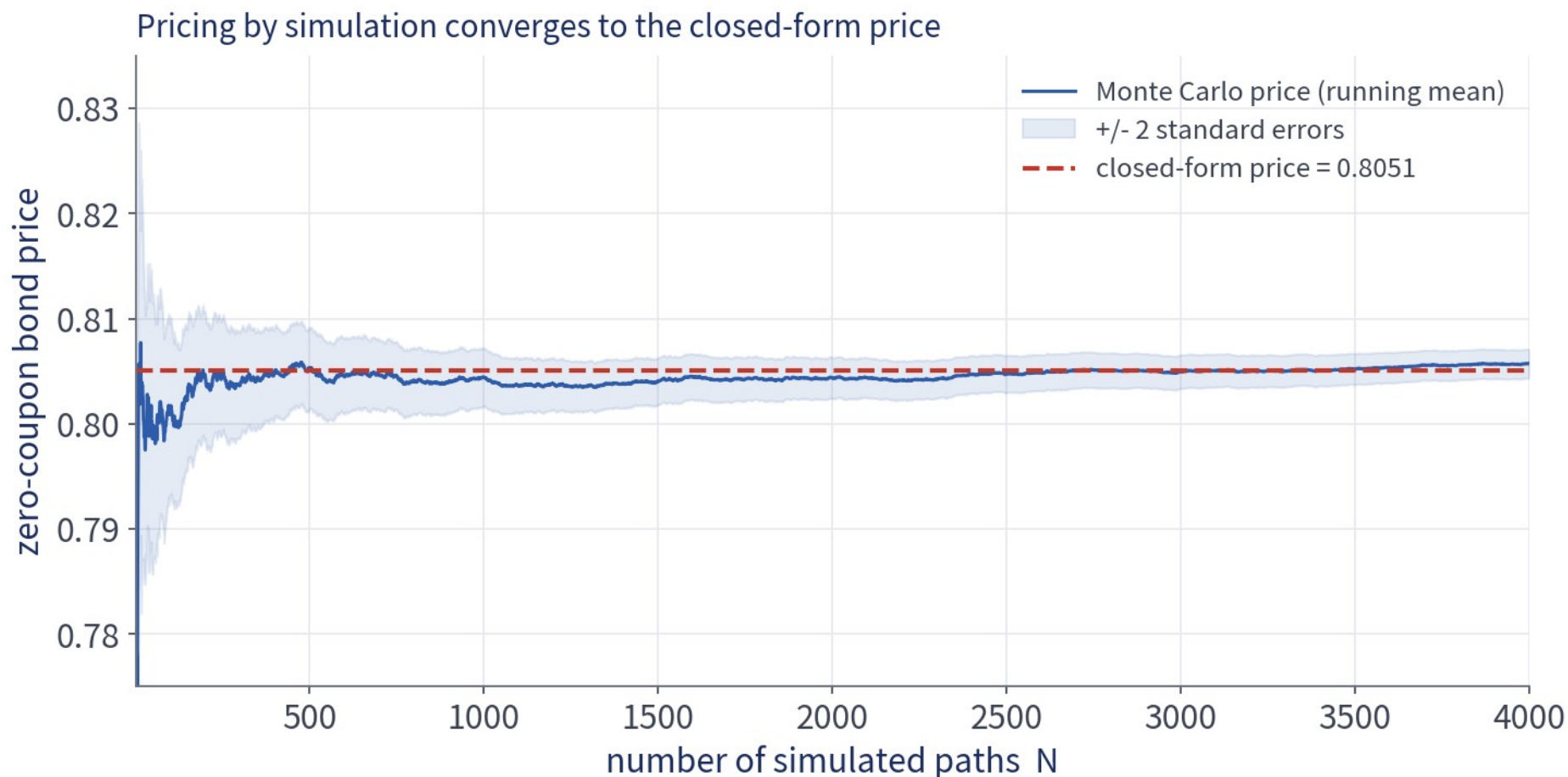
Average across paths

The bond price is the mean discount factor over all paths. More paths shrink the standard error, tightening the estimate.

The price is just an average of discounted payoffs — the pricing expectation, made numerical.

Simulation Converges to the Closed Form

The Monte Carlo bond price, as a running average over simulated paths, converges to the analytic Vasicek price. The band of two standard errors narrows as the number of paths grows.

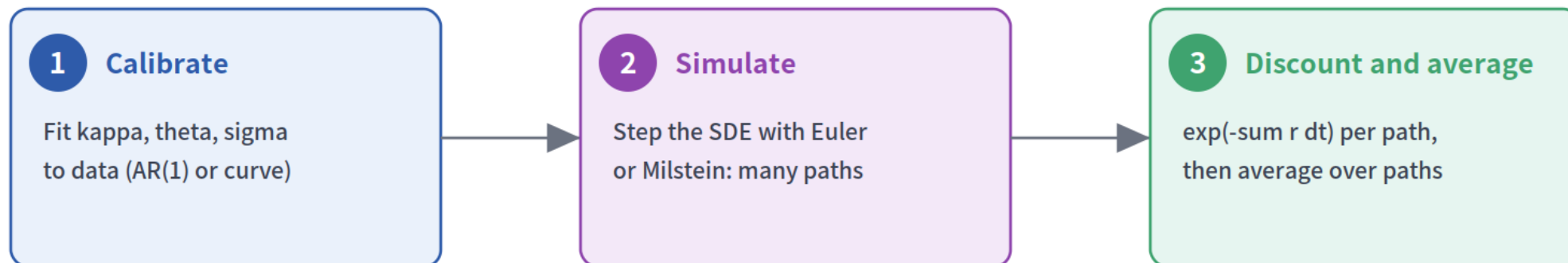


The End-to-End Pricing Pipeline

Calibrate the parameters, simulate paths by Euler or Milstein, then discount and average into a price. Always cross-check against the closed form before trusting the engine on products that lack one.

The End-to-End Pricing Pipeline

Three stages turn raw data into a bond price: calibrate, simulate, discount-and-average.



Always cross-check against the closed form

For Vasicek and CIR you can price a zero-coupon bond analytically. Use that to VALIDATE the simulator: the Monte Carlo price should converge to the closed-form price as the number of paths grows.

Simulation shines for products with no closed form - once the engine is trusted.

Simulate or Use the Formula?

Match the tool to the task

Use the closed form when

- You are pricing a plain bond under Vasicek or CIR.
- Speed and exactness matter.
- A formula exists — there's no reason to simulate.

Simulate when

- The product has no closed-form price.
- Payoffs are path-dependent or exotic.
- You need a flexible, general-purpose engine.

Calibrate and Price a Zero-Coupon Bond

Direct Claude Code through the full pipeline

GOAL Calibrate a short-rate model, price a bond by simulation, and check it against the closed form.

[INPUT] Prompt · Claude Code

Write and run `price_bond.py` that:

1. estimates Vasicek κ , θ , σ from a short-rate series via the AR(1) regression;
 2. simulates many short-rate paths with Euler and discounts each: $\exp(-\sum r \cdot dt)$;
 3. prices a zero-coupon bond as the average discount, with a standard error;
 4. compares to the closed-form Vasicek price and plots convergence as the path count grows.
- Confirm the simulated price converges to the closed form within the error band.

How to work with it

- AR(1) gives the parameters.
- Discount = $\exp(-\sum r \cdot dt)$.
- More paths shrink the error.
- It should match the closed form.

Deliverable: `price_bond.py` + a calibrated model, an MC bond price, and a closed-form check.

Three Questions Before You Finish

If you can answer these, Week 12 has landed

Q1	In the Euler step, the random shock scales with what power of dt ?	The square root
Q2	What does Milstein add over Euler?	A second-order correction
Q3	How do you validate a simulated bond price?	Against the closed form

All three come from the Euler step, the Milstein correction, and the closed-form check.

Three Pitfalls to Avoid

Where the numerics go wrong

Common mistakes

- "A big time step is fine — it's roughly right."
- "Naive Euler is safe for CIR."
- "A simulated price needs no validation."

The reality

- Large steps add real error; check sensitivity to dt .
- Euler can push CIR negative — truncate or use the exact scheme.
- Always cross-check against the closed form when one exists.

Week 12 in Three Ideas

Pulling the whole week together

1

Rates revert — Vasicek and CIR

Parts 1-2: the short rate is a mean-reverting SDE. Vasicek is OU for the rate; CIR keeps it positive with a square-root diffusion. Both are affine, giving closed-form bond prices and whole yield curves.

2

Discretize to compute

Euler-Maruyama steps the SDE; Milstein adds a correction for faster convergence. Mind the pitfalls — discretization error and CIR's zero boundary.

3

Calibrate, simulate, and validate

Fit the parameters to history or the curve, price a bond by simulating and averaging discounted payoffs, and check the result against the closed form.

Next — Week 13: Bayesian inference, treating those calibrated parameters as distributions, not points.